

ClaudeClaw v2 Visual Guide

Every diagram from the ClaudeClaw v2 video, organized by section. Each image includes a plain-English caption explaining what you are looking at and how it fits into the system. Use this as a reference while building, or as a standalone overview of the architecture.

by Mark Kashef

Early AI Dopters

Contents

- 01** Setup & The Reframe
- 02** Features & Payoff
- 03** Why This Moment Matters
- 04** Build Your Own War Room
- 05** The Full Stack

01

Setup & The Reframe

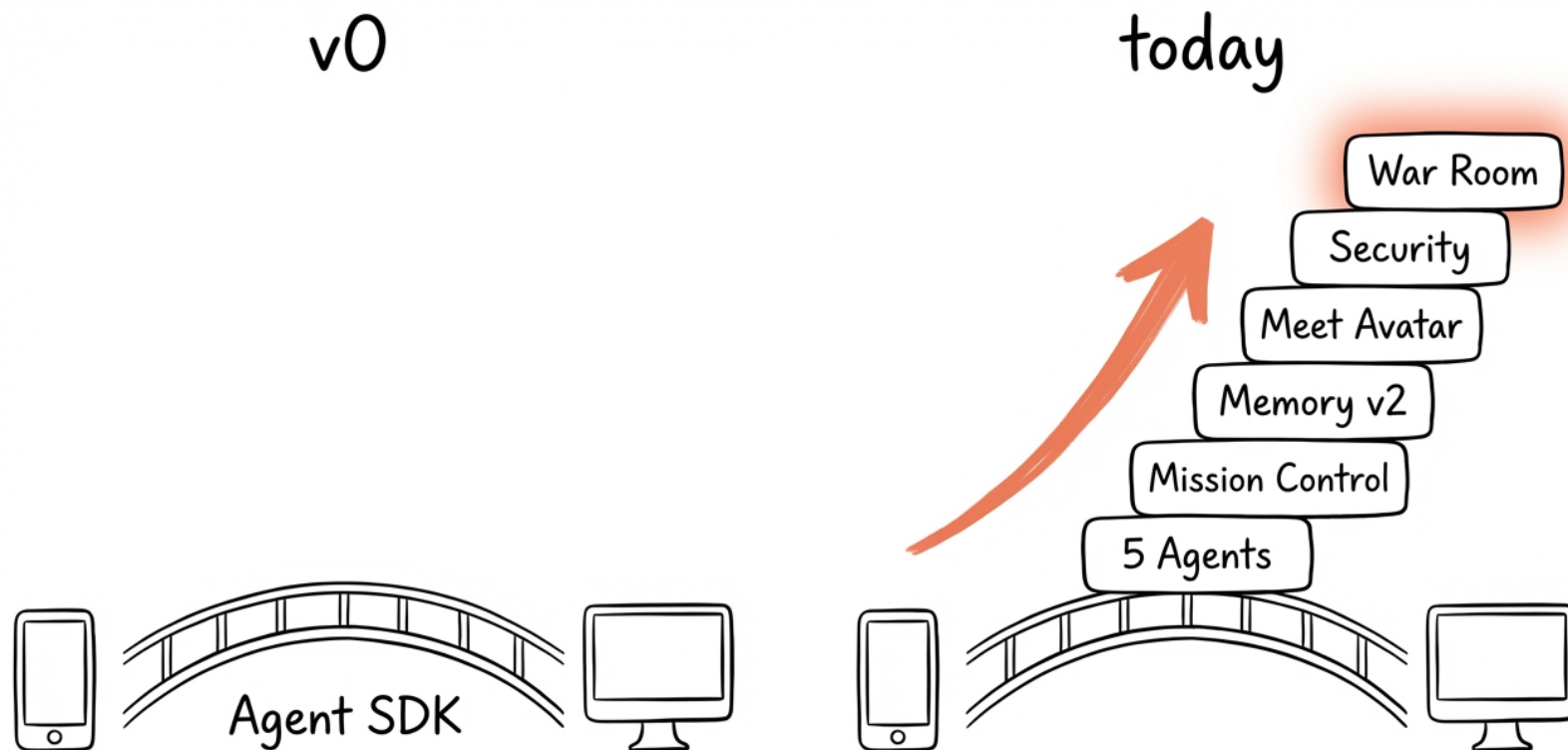
The foundation. How ClaudeClaw bridges your phone to your desktop, what changed since v0, and why this approach beats every alternative.

The Bridge



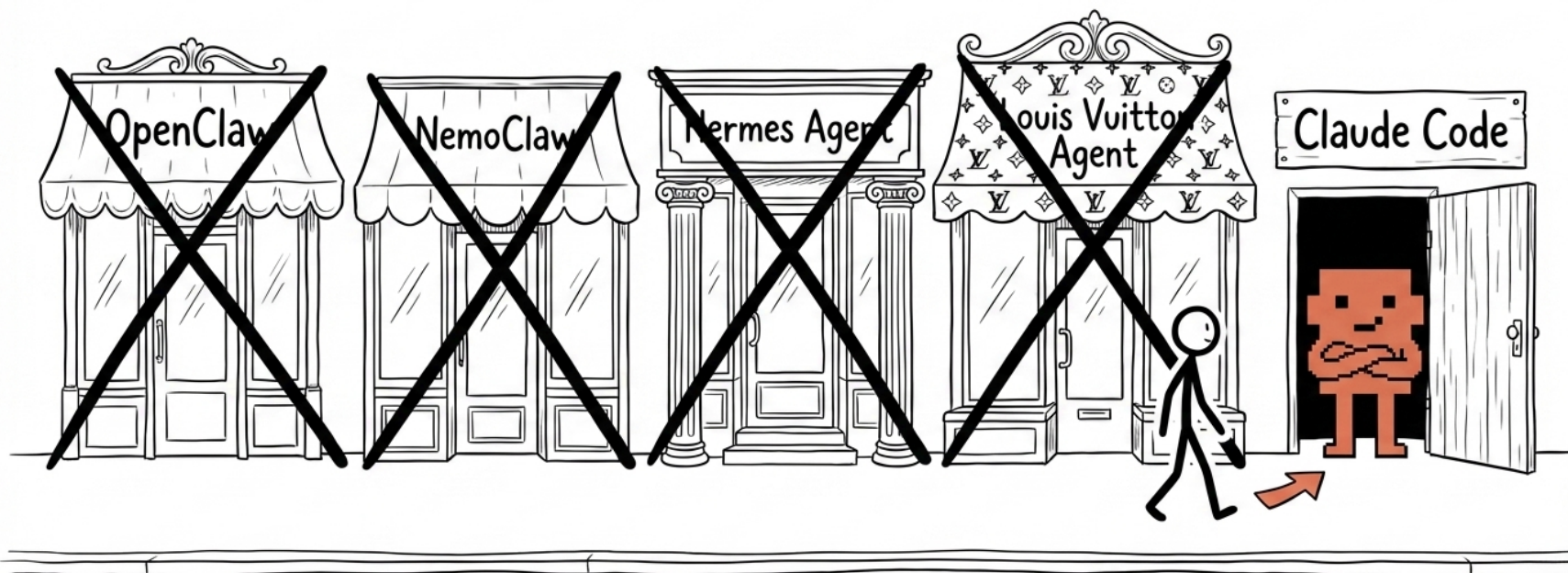
Your phone talks to your Mac through a thin bridge called the Agent SDK. It spawns a real Claude Code session on your desktop and sends the result back. One brain, not two.

100 Plus Changes Later



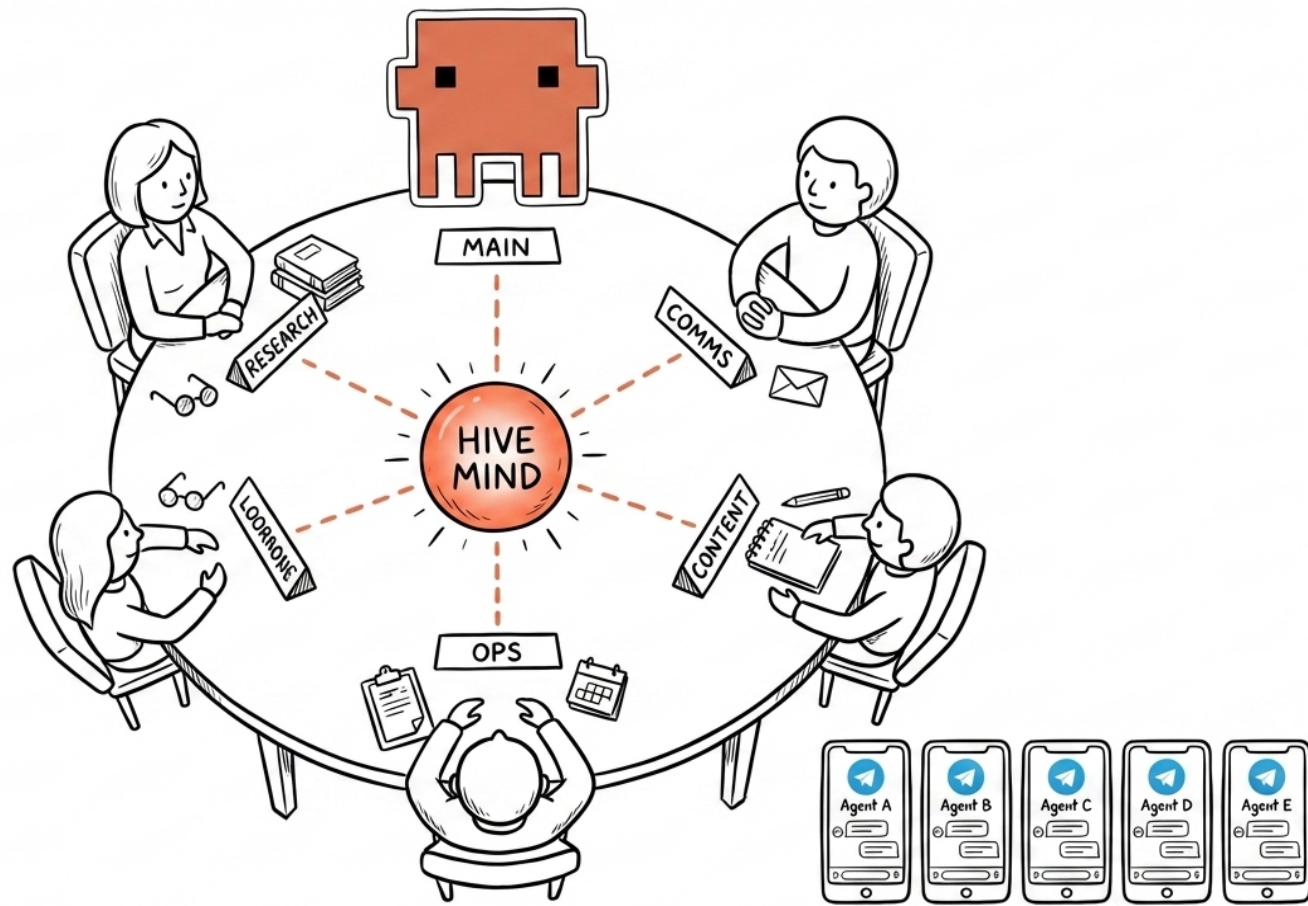
v0 was a bridge. Today, six layers sit on top of it: five specialized agents, mission control, memory v2, a video meeting avatar, a security stack, and the War Room. The bridge underneath never changed.

Not OpenClaw. Not Hermes. Not Louis Vuitton.



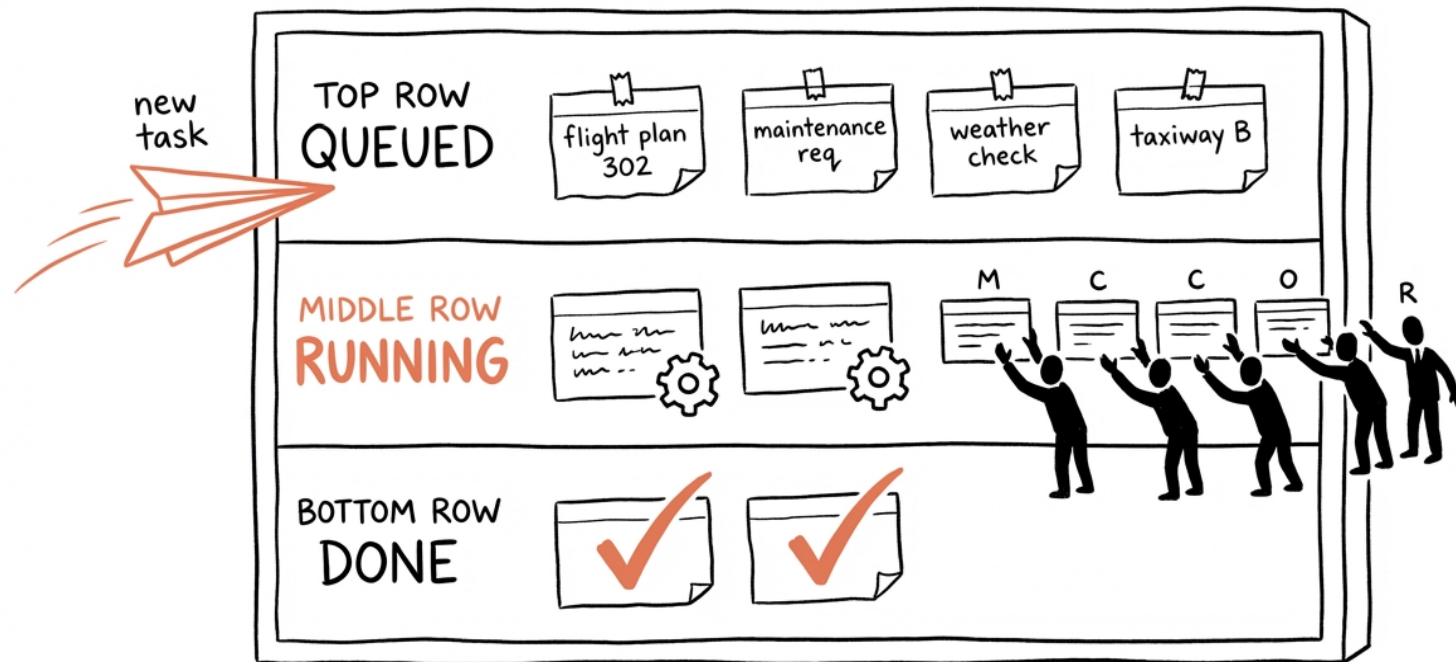
Every week brings a new framework. OpenClaw, NemoClaw, Hermes Agent, and yes, Louis Vuitton Agent (that one is a joke). The workshop at the end of the row is Claude Code. With layers.

The Team



Five specialist agents share a round table. Main handles triage. Comms runs your inbox. Content handles YouTube and LinkedIn. Ops manages billing and calendar. Research does deep web dives. They share a Hive Mind notebook in the center.

Mission Control



Tasks land in the queue, get auto-assigned to the right agent, move to running, then done. Like air traffic control for your AI workforce. The paper airplane is your next task flying in.

The War Room



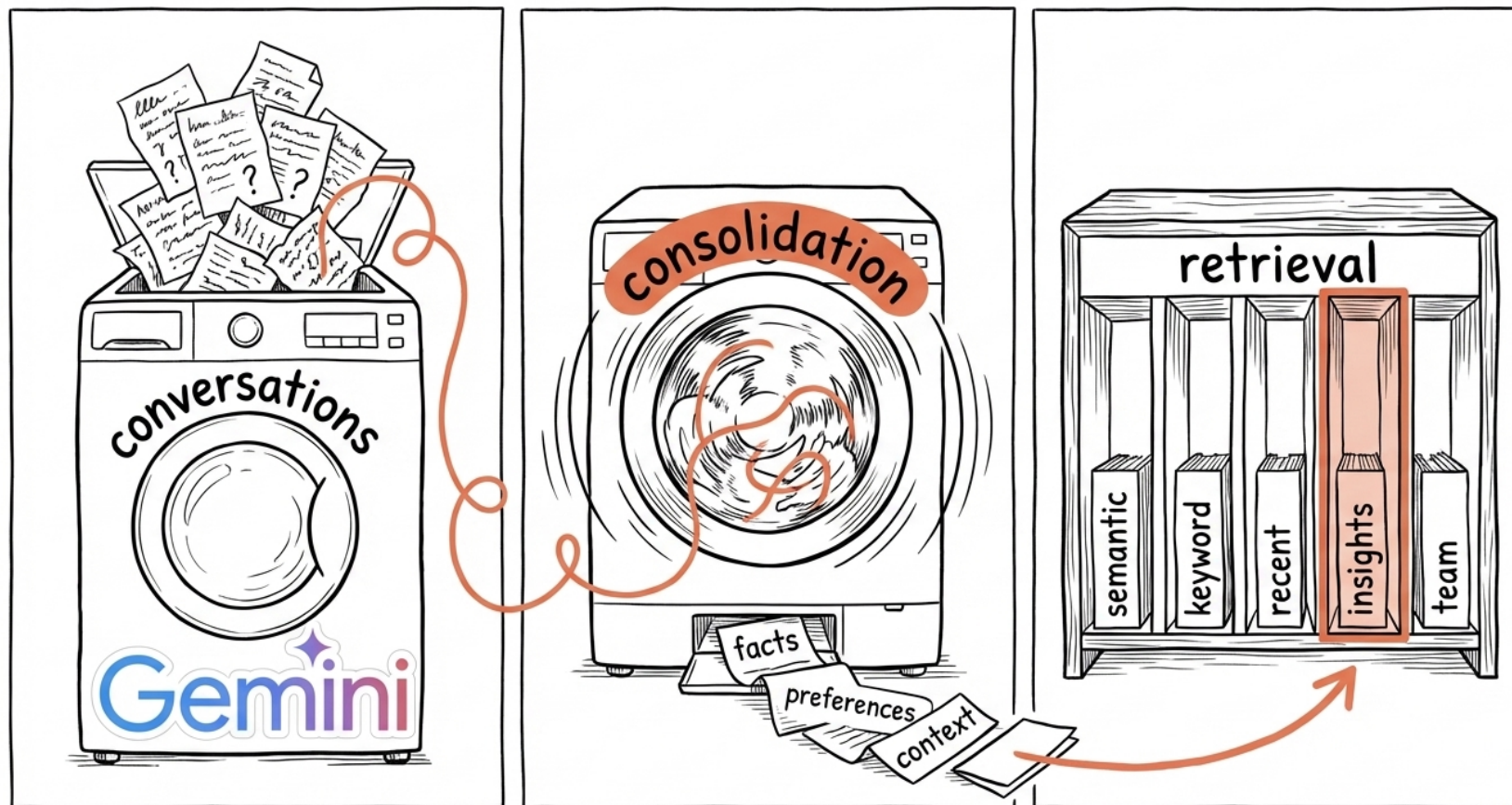
A live voice room where all five agents sit around a conference table. Speak into the room, the router picks who answers, and they reply in their own voice. The coral ring in the center is the voice router.

02

Features & Payoff

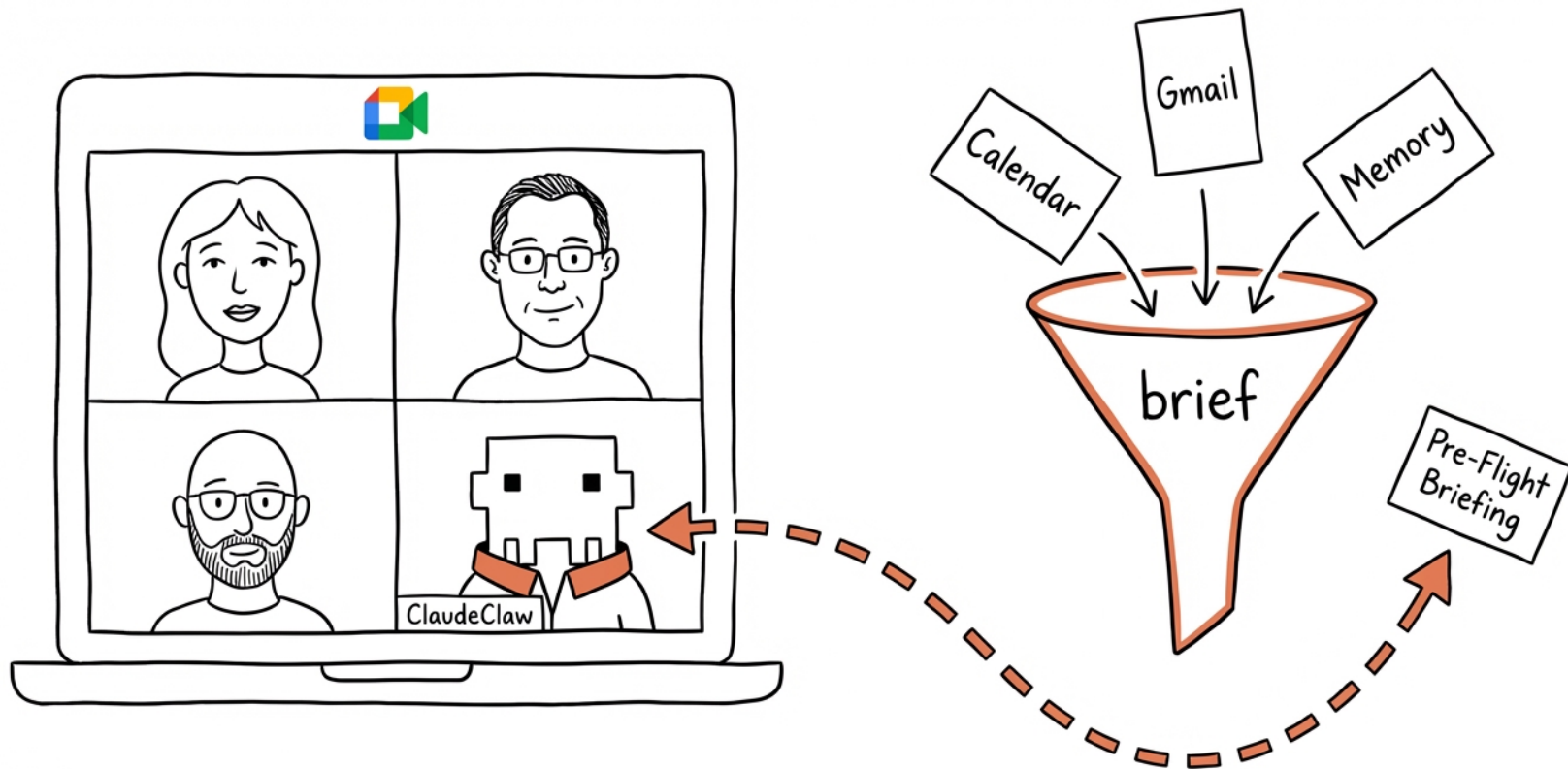
What the system actually does for you every day. Memory, meetings, channels, security, and the pipeline under the hood.

Memory That Improves Itself



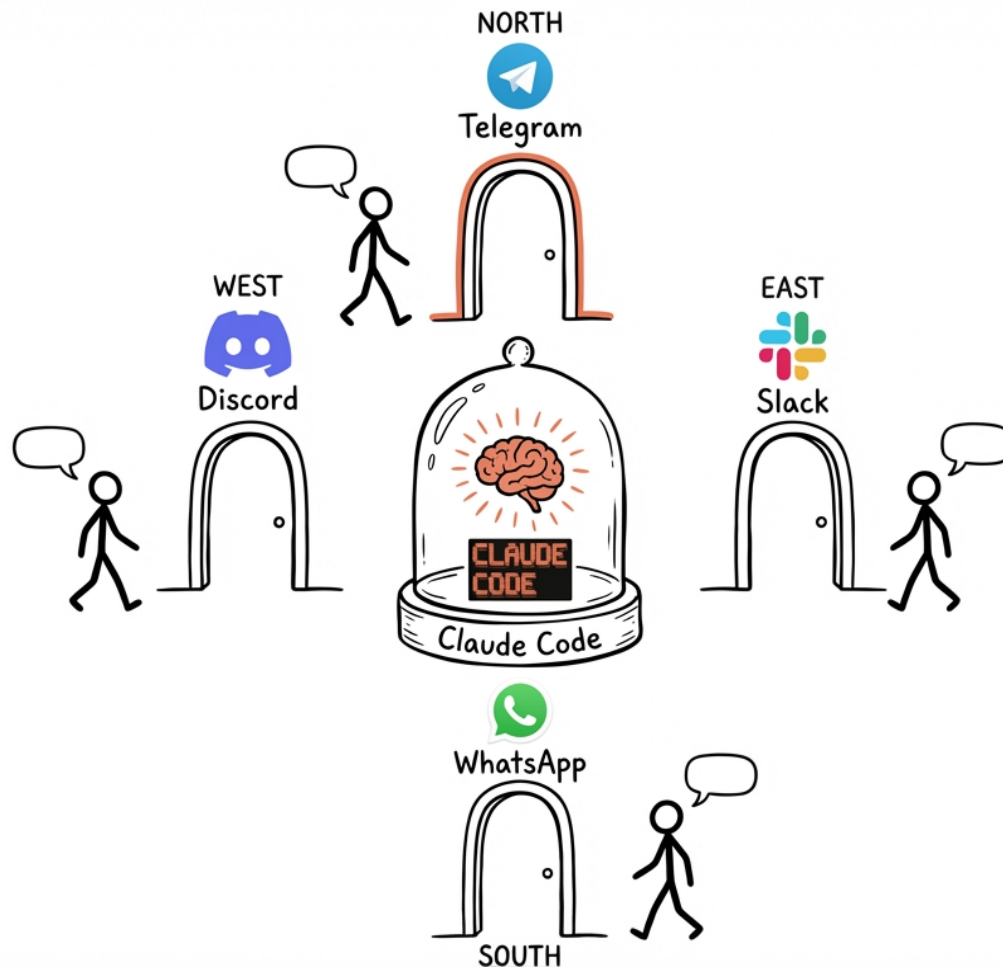
Conversations go into a washing machine. A Gemini-powered process extracts the facts worth keeping, folds them into a five-section bookshelf: semantic, keyword, recent, insights, and team. The insights shelf notices patterns across your conversations.

The Meeting Assistant



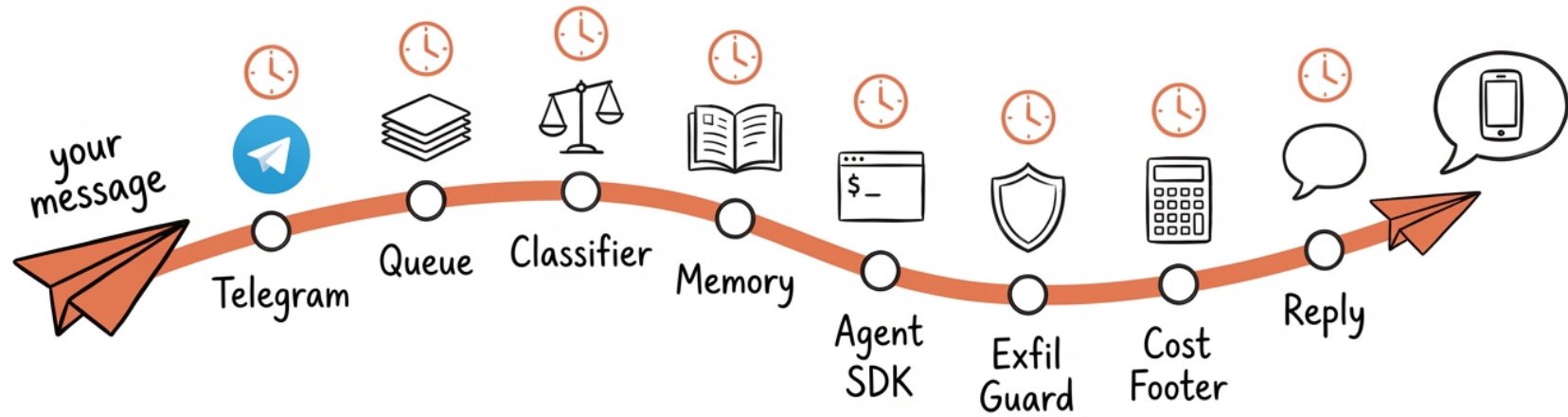
Your agent joins a real Google Meet as a video avatar. Before it walks in, it pulls your Calendar, Gmail, and Memory to build a 75-second pre-flight briefing. By the time it appears on screen, it knows who everyone is.

Pick Your Channel



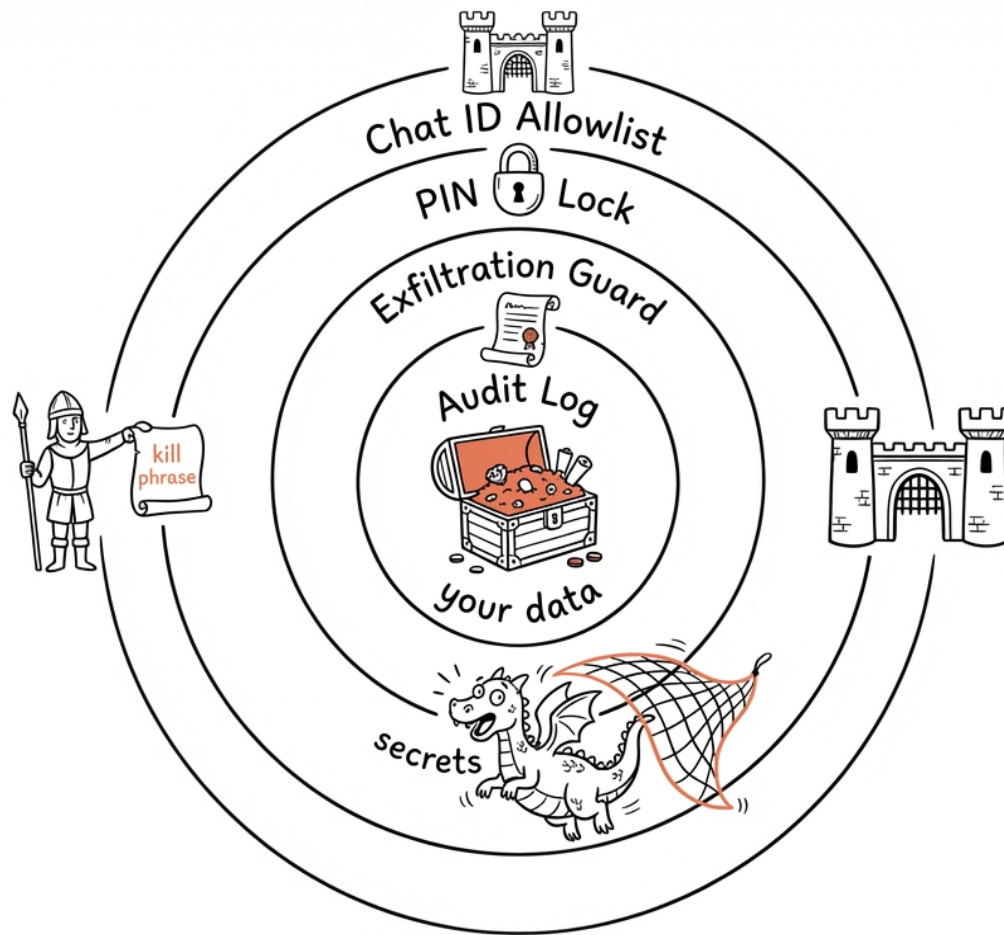
The brain in the jar is Claude Code. Around it, four doorways: Telegram, Slack, WhatsApp, Discord. Pick the one you use. The bridge does not care which doorway you walk through.

One Message, Many Steps



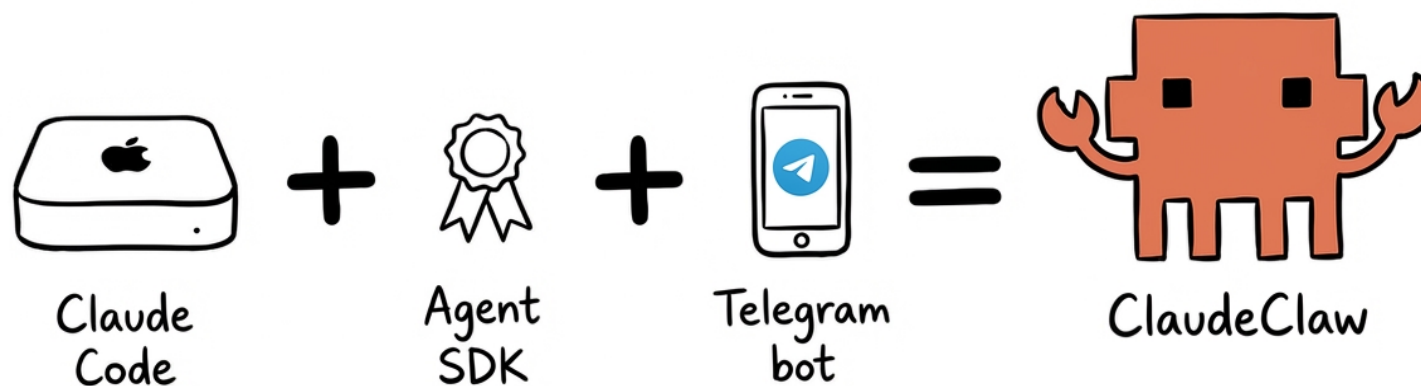
A single message passes through eight stages in under five seconds: Telegram, queue, classifier, memory injection, Agent SDK, exfiltration guard, cost footer, and reply. Each station adds something before Claude sees your message.

Guarded on Every Side



Four concentric rings of defense. Outer ring: a guest list of approved Telegram accounts. Then a PIN lock. Then an exfiltration guard that catches leaked secrets. At the center, a treasure chest labeled 'your data.' A guard holds the emergency kill phrase.

What You Need



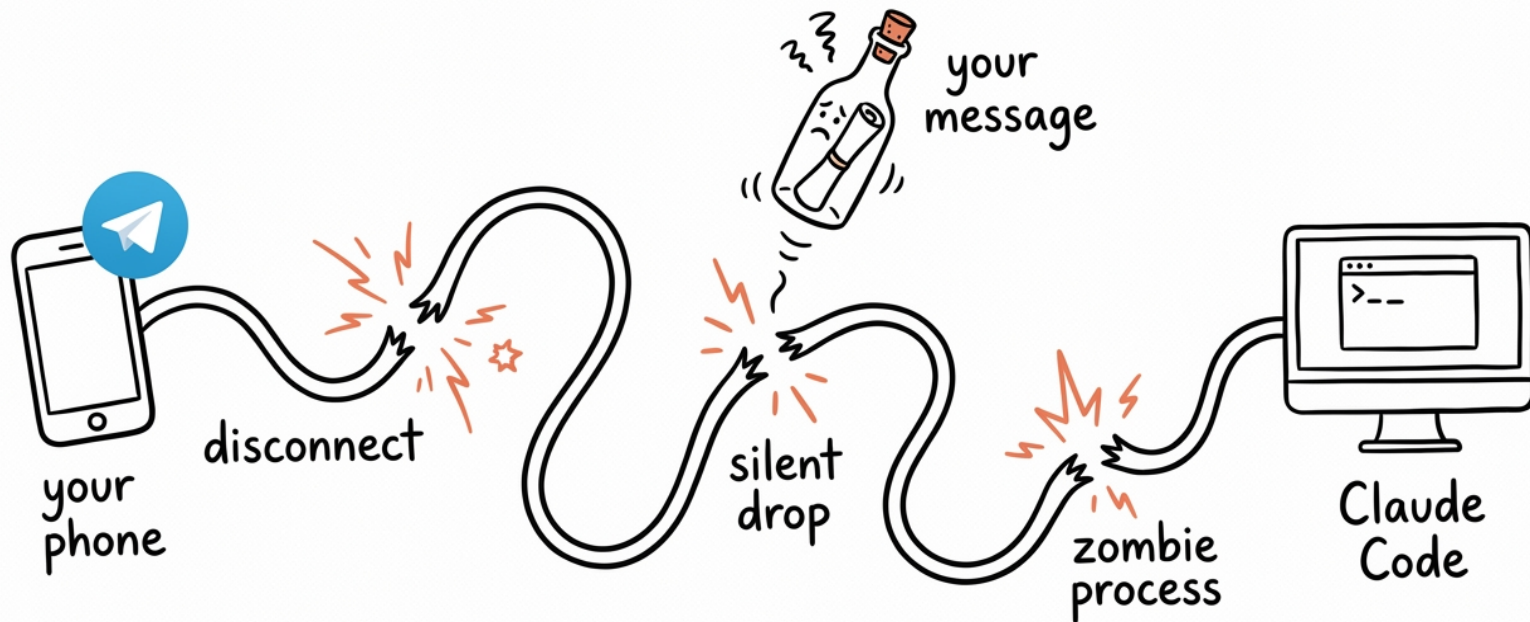
Three things: Claude Code on your Mac, the Agent SDK (a free toolkit from Anthropic), and a Telegram bot token from BotFather. You probably already have most of this.

03

Why This Moment Matters

The competitive landscape, the Anthropic policy story, and the emotional thesis. Why this specific approach is the one that lasts.

Why Not Native Channels?



Channels keeps cutting out

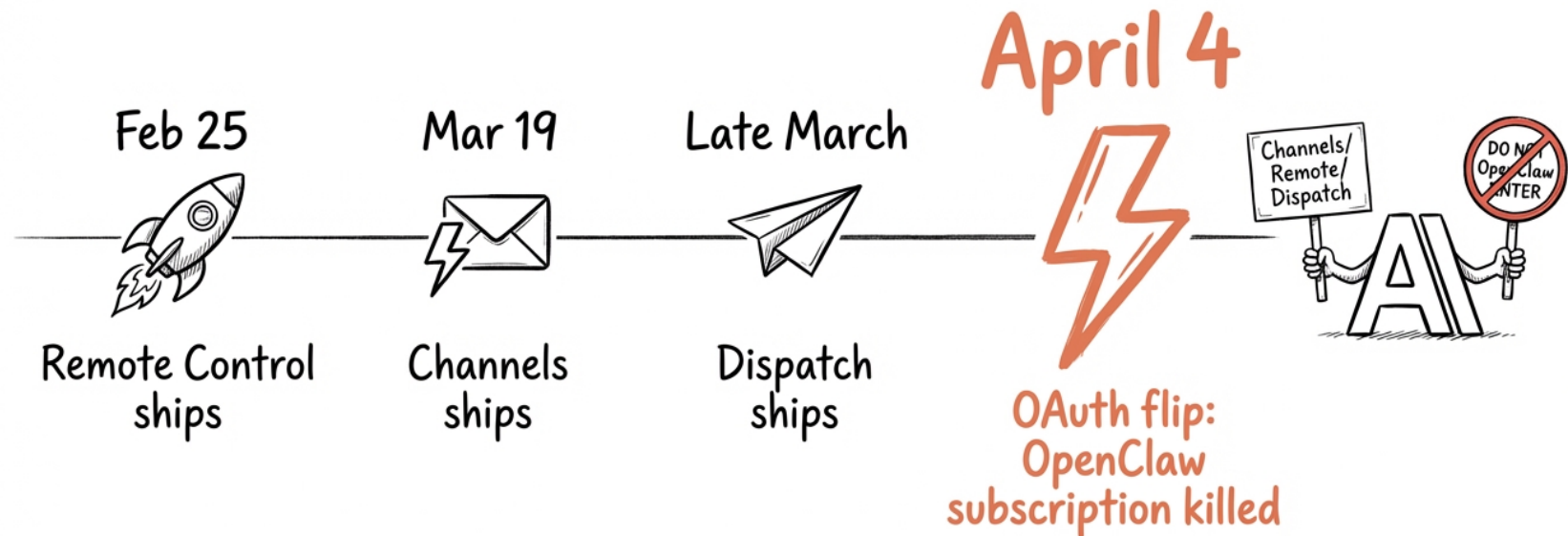
Anthropic shipped their own Telegram integration called Channels. The phone line keeps cutting out. Six open GitHub issues describe the same bug: inbound messages silently drop because the Telegram plugin and Claude Code have a protocol-level mismatch.

How This Stacks Up

	 ClaudeClaw	Channels	Remote Control	Dispatch
Runs 24/7	✓	✗	✗	✗
Works Offline Terminal	✓	✗	✗	✗
Chat-First	✓	✓	✗	✗
Team & Enterprise	✓	✗	✗	✗

A four-column comparison: ClaudeClaw vs Channels vs Remote Control vs Dispatch. ClaudeClaw wins every row: runs 24/7, works offline, chat-first, and available on Team and Enterprise plans.

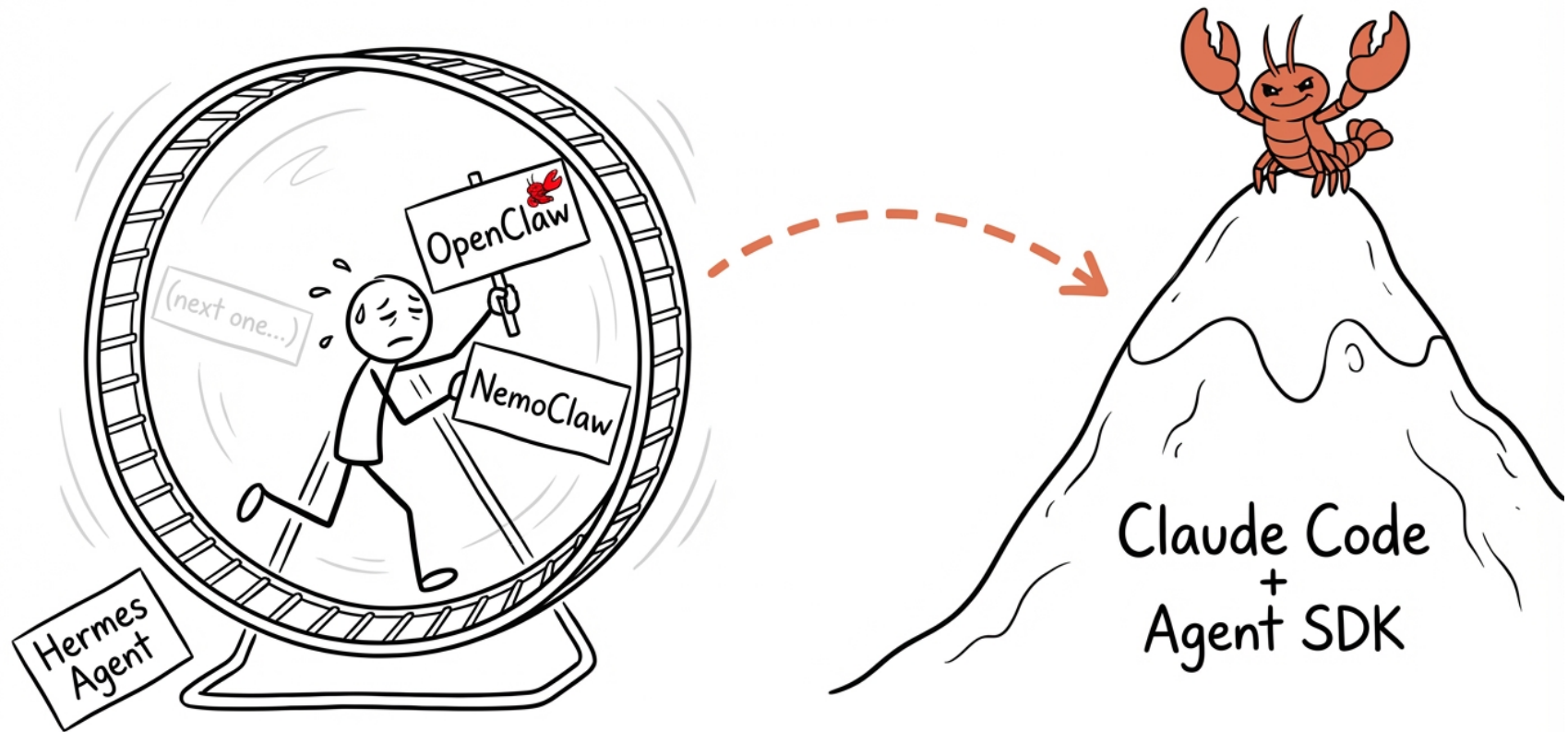
The Sequence Nobody Noticed



First they shipped their own. Then they shut the rest down.

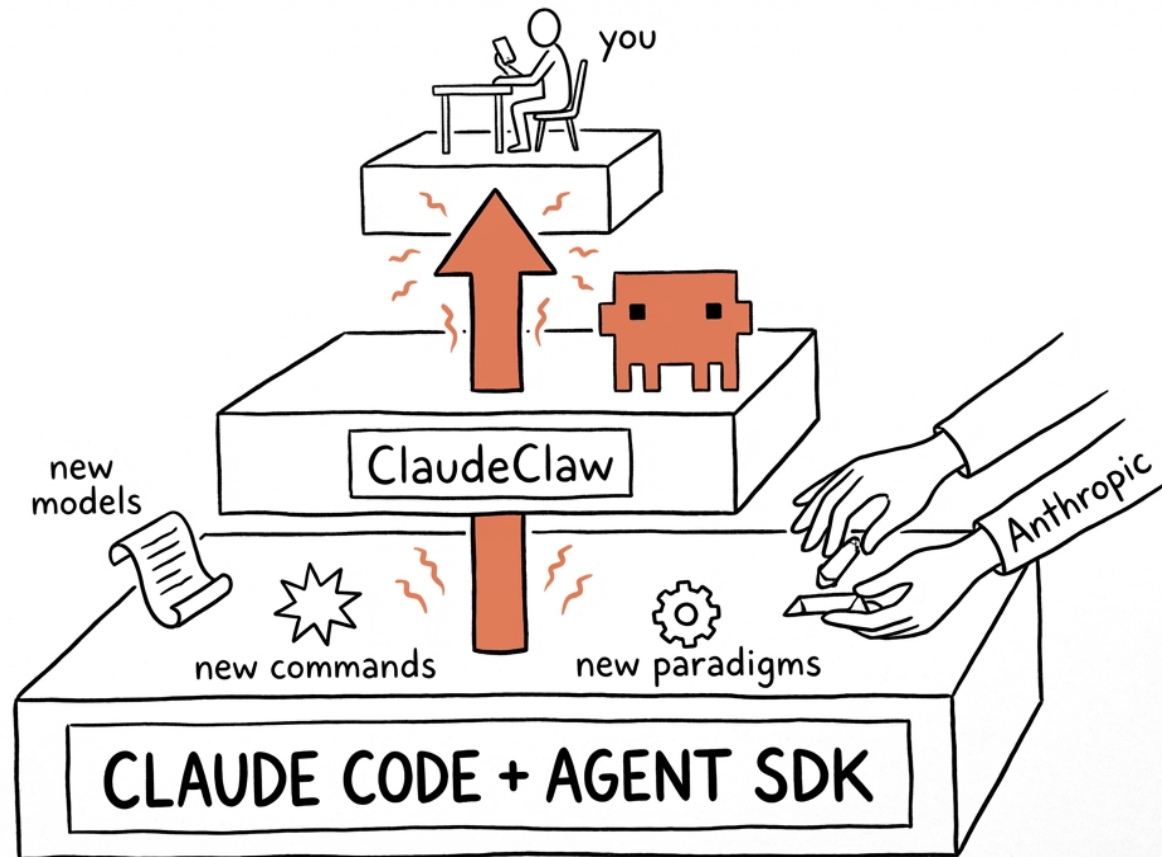
Feb 25 Anthropic shipped Remote Control. March 19 they shipped Channels. Late March, Dispatch. April 4, the billing flip that killed OpenClaw's subscription access. First they shipped their own, then they shut the rest down.

Stop Running on the Treadmill



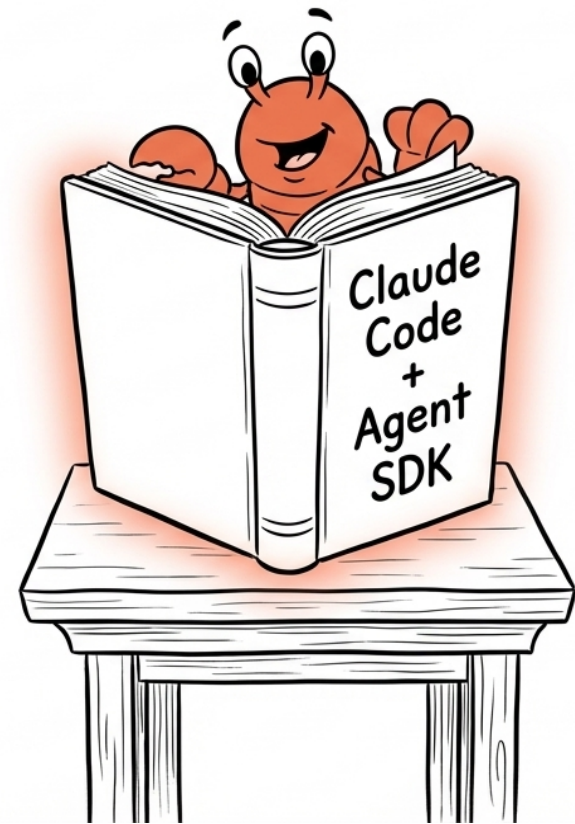
A tired figure runs on a hamster wheel labeled with framework names. On a mountain beside them, the Clawd mascot stands on solid ground labeled Claude Code and Agent SDK. The escape arrow traces the path from the wheel to the peak.

It Gets Better While You Sleep



A three-tier pyramid. Anthropic ships new features into the bottom tier. They flow up through ClaudeClaw in the middle tier to the user on top. When Anthropic ships, your setup improves automatically.

Learn One Thing. Once.



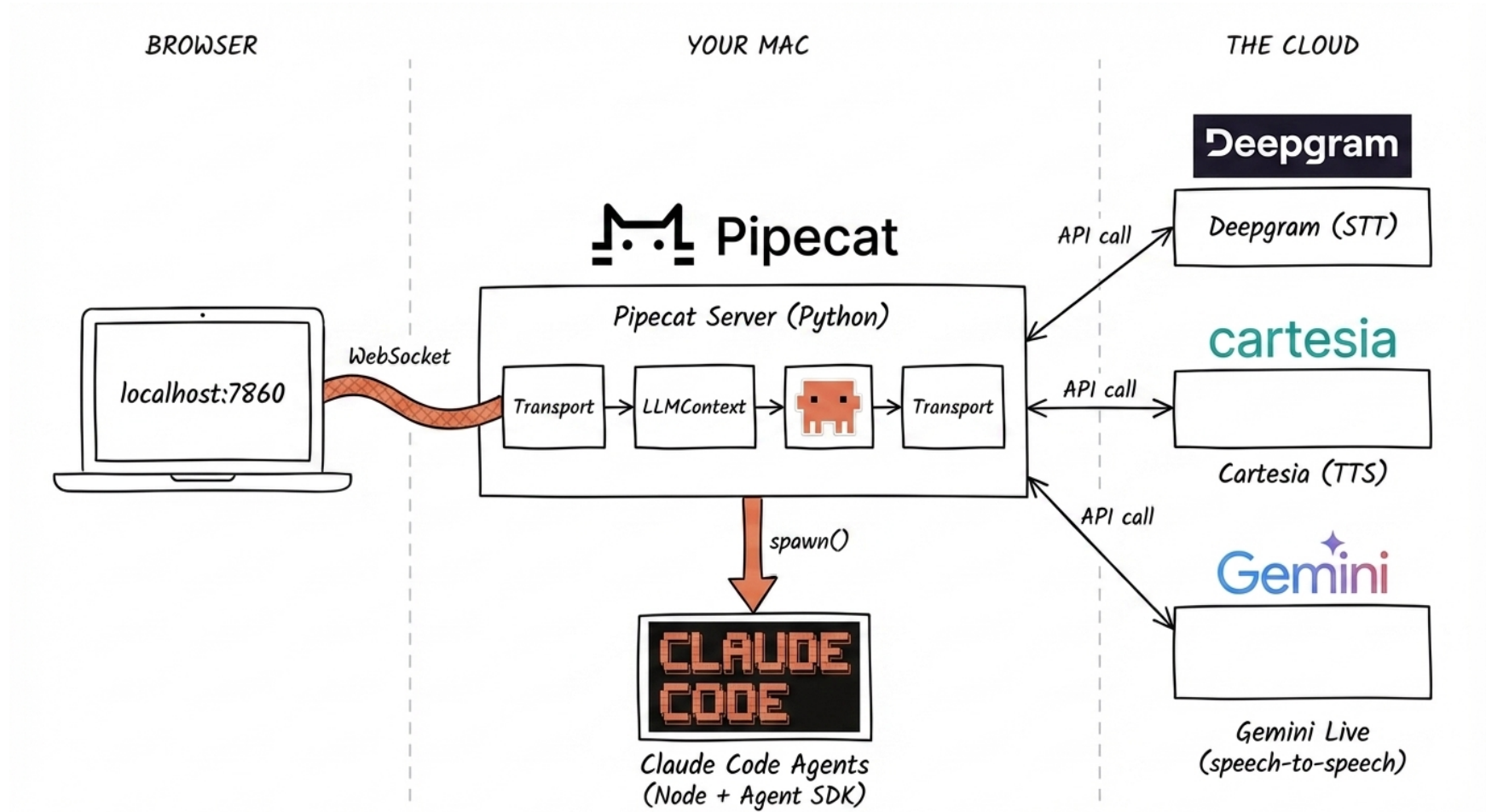
A chaotic pile of expired framework pamphlets on the left. A single thick book titled Claude Code plus Agent SDK on the right, with the Clawd mascot reading happily on top. Pick one thing. Learn it once. Build on it forever.

04

Build Your Own War Room

The reproducible build guide. Every component, every frame type, every API call. Follow these diagrams with the video transcript and you can rebuild it yourself.

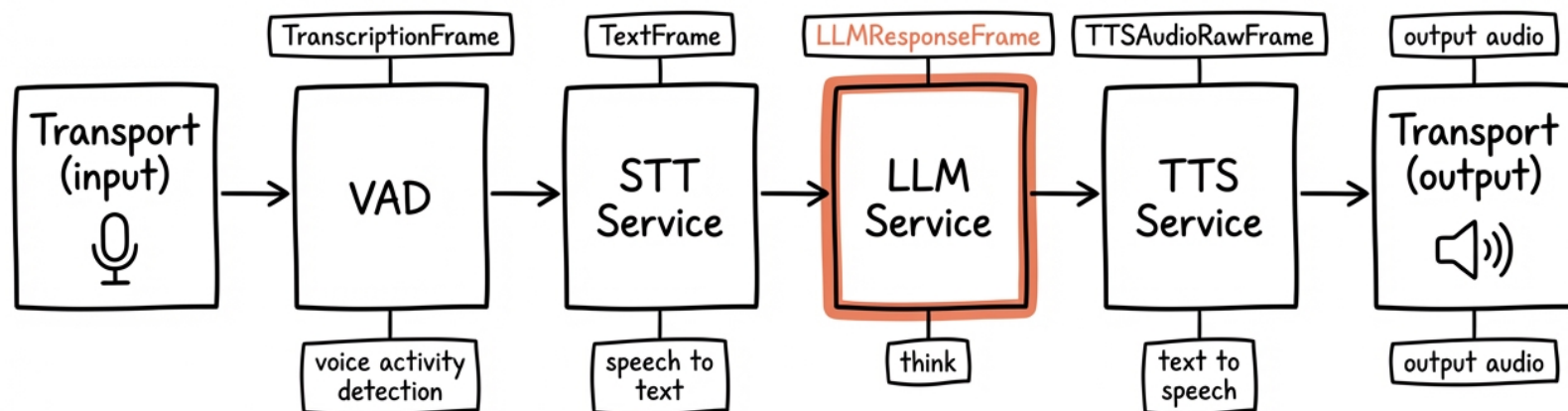
The Map



Three zones. Your browser on the left connects via WebSocket to the Pipecat server on your Mac in the middle. The server calls three cloud services on the right: Deepgram for listening, Cartesia for speaking, and Gemini Live for thinking. Below, a subprocess arrow drops to your Claude Code agents.

Pipecat, Plainly

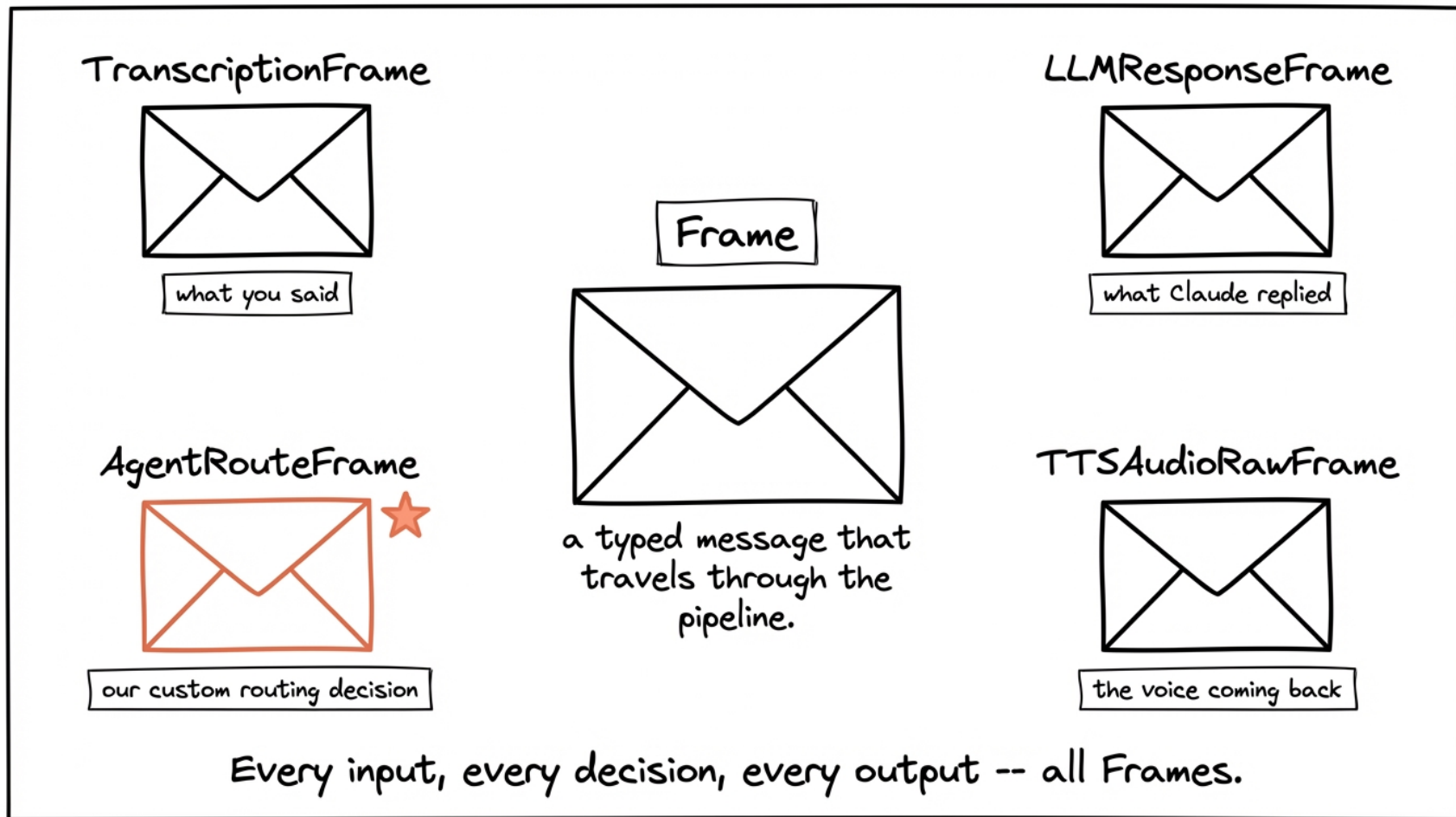
Pipecat



Every event is a Frame. Frames flow left to right through processors.

A six-station pipeline. Audio enters at Transport in, passes through VAD (voice activity detection), STT (speech to text), LLM (the decision center, highlighted in coral), TTS (text to speech), and out through Transport. Every event moving through the pipeline is a typed message called a Frame.

What Is a Frame



A Frame is like an envelope. Four examples surround a central Frame icon: TranscriptionFrame carries what you said, LLMResponseFrame carries what Claude replied, TTSAudioRawFrame carries the voice coming back, and AgentRouteFrame (marked with a coral star) is the custom routing decision you define yourself.

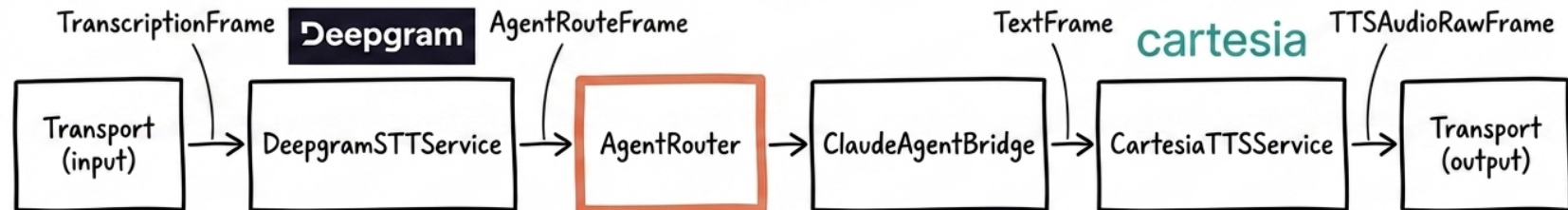
Two Modes

LIVE MODE



one service. google handles speech-to-speech.

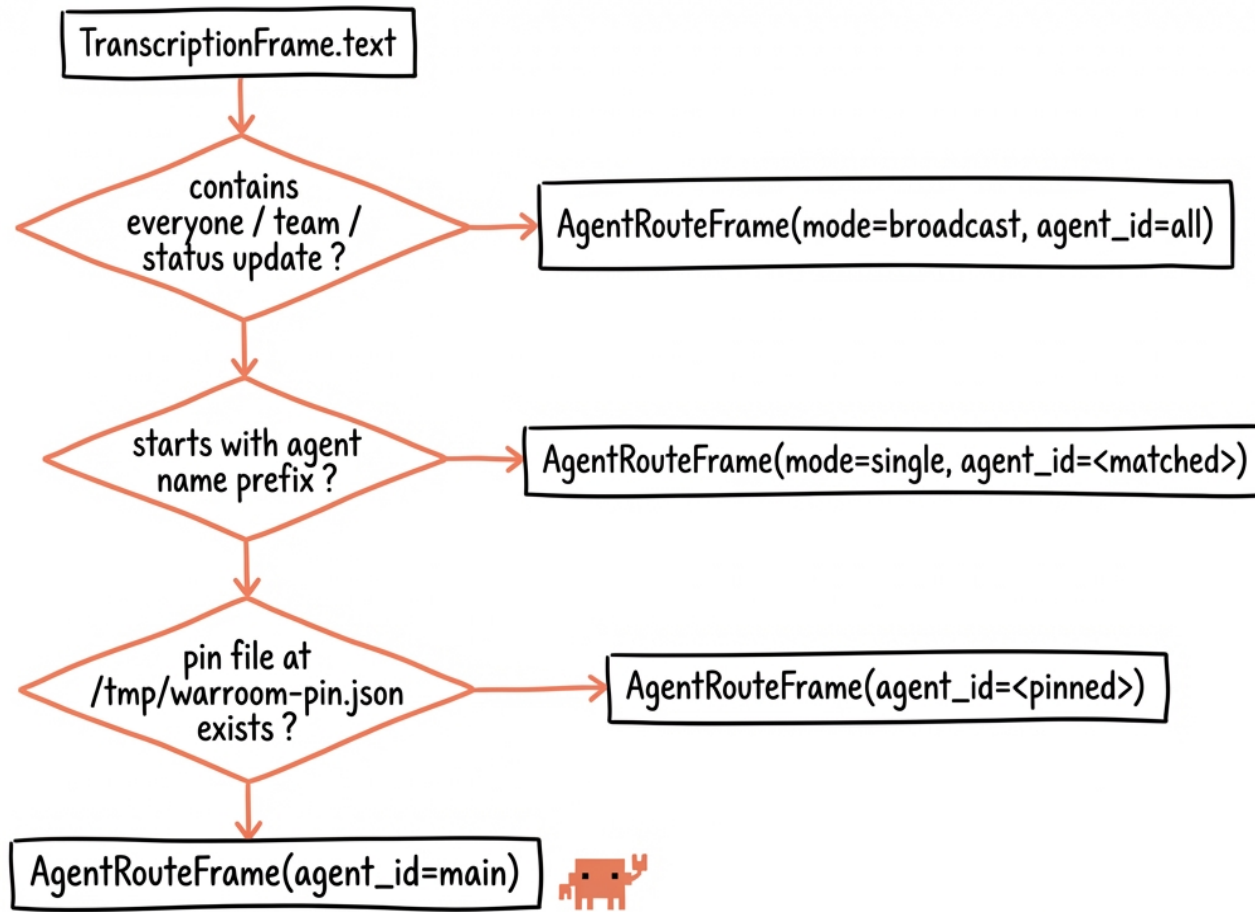
LEGACY MODE



chain of services. you pick each one.

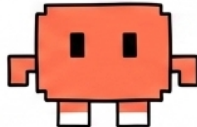
Top row is Live Mode: one box (Gemini Live) does everything, lowest latency. Bottom row is Legacy Mode: a chain of three services (Deepgram for ears, AgentRouter for brain, Cartesia for voice) gives you more control. Pick based on simplicity vs customization.

The Router



Three decision diamonds in sequence. Does the message contain 'everyone' or 'team'? Broadcast to all agents. Does it start with an agent name? Route to that agent. Is there a pin file? Route to the pinned agent. Otherwise, default to Main.

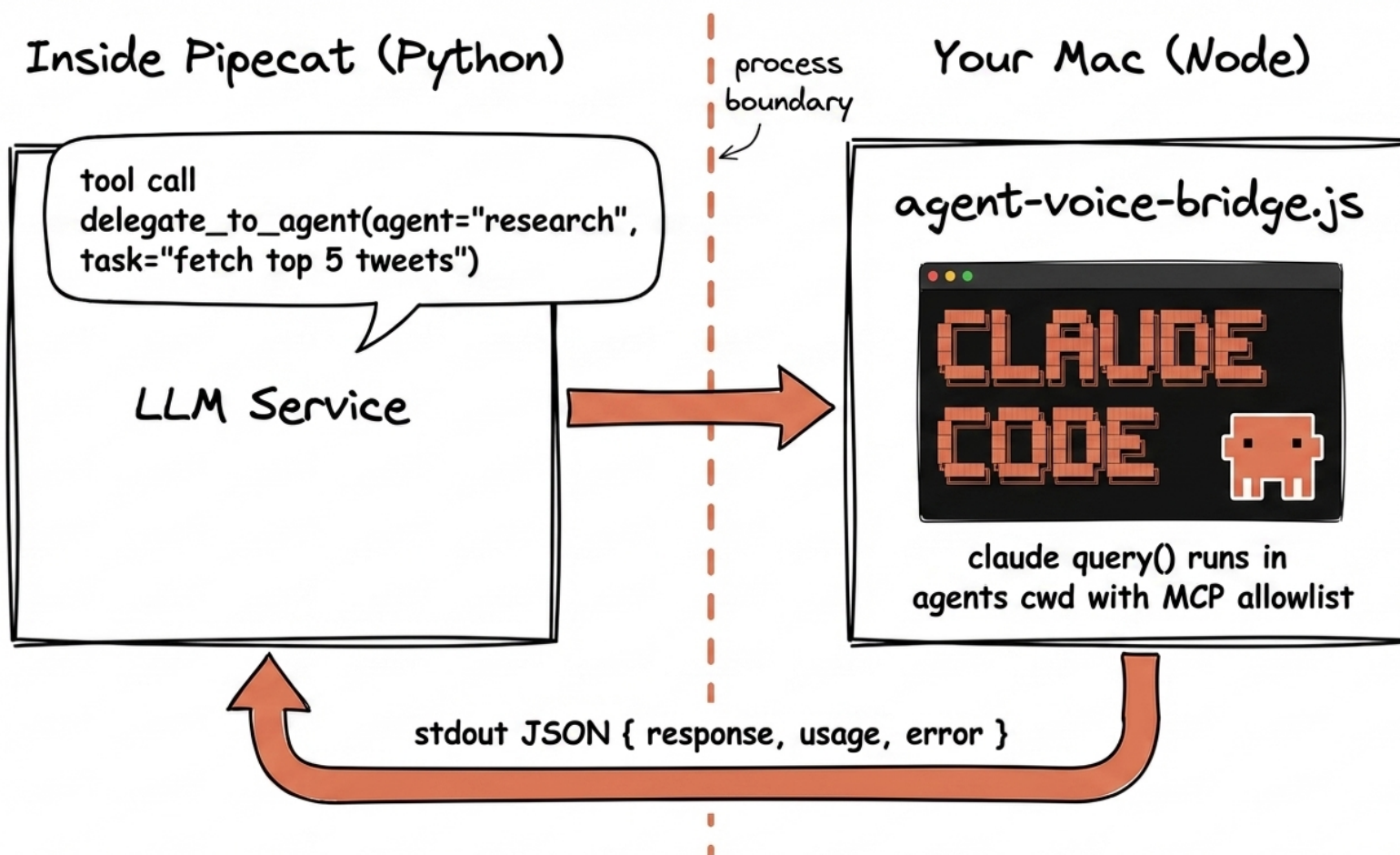
One Persona

research	
name: “research”	
cartesia_voice_id: “bf991597-6c13-47e4-8411...”	
gemini_voice: “Kore”	
system_prompt: “You are Research, the Grand Maester. Lead with the conclusion, cite sources, flag confidence...”	
tools: delegate_to_agent answer_as_agent get_time list_agents	

warroom/personas.py

A spec card for the Research agent showing five fields: name, Cartesia voice ID, Gemini voice, system prompt, and tools. Copy this card, change the voice and personality, and you have a new agent.

Crossing the Process Boundary



Two panels divided by a dashed line labeled 'process boundary.' On the left, the Pipecat voice room calls a tool. On the right, a Node.js subprocess runs a real Claude Code agent on your Mac. The return path at the bottom carries the JSON result back to the voice room.

What You Need (Build)

SERVICES

Deepgram

DEEPGRAM_API_KEY

real-time STT

cartesia

CARTESIA_API_KEY

per-agent TTS

Gemini

GOOGLE_API_KEY

live mode speech-to-speech

Daily.co

DAILY_API_KEY

(optional) for multi-user rooms

INSTALL & RUN

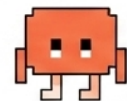
```
Terminal

pip install
pipecat-ai[websocket,deepgram,
deepgram,cartesia,silero]

python warroom/server.py

open localhost:7860
```

localhost:7860



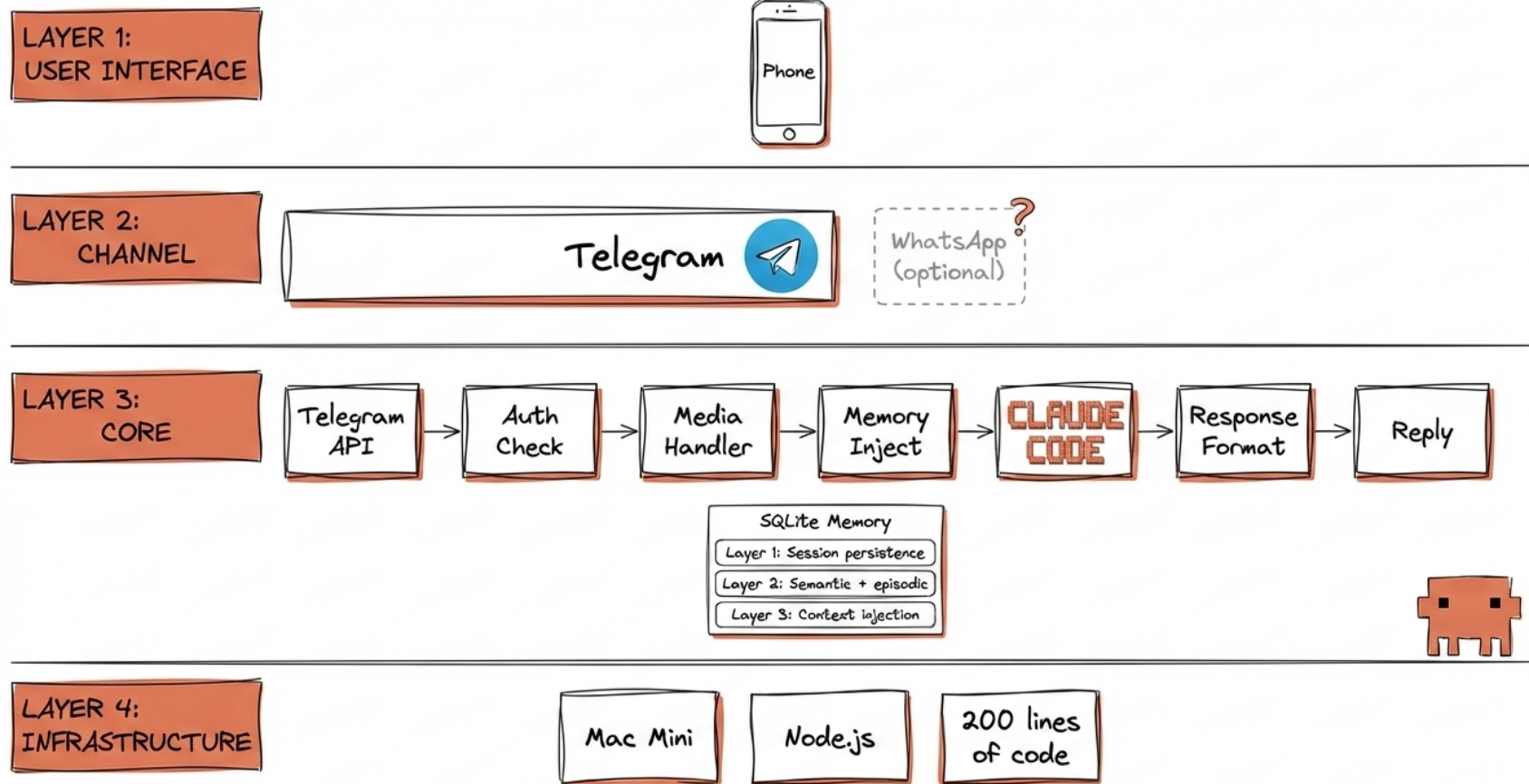
Four services on the left: Deepgram, Cartesia, Gemini, and Daily.co (optional). Each with its API key. On the right, a terminal window showing three commands: pip install, start the server, open localhost 7860.

05

The Full Stack

The complete before-and-after. What v0 looked like versus what v2 looks like today.

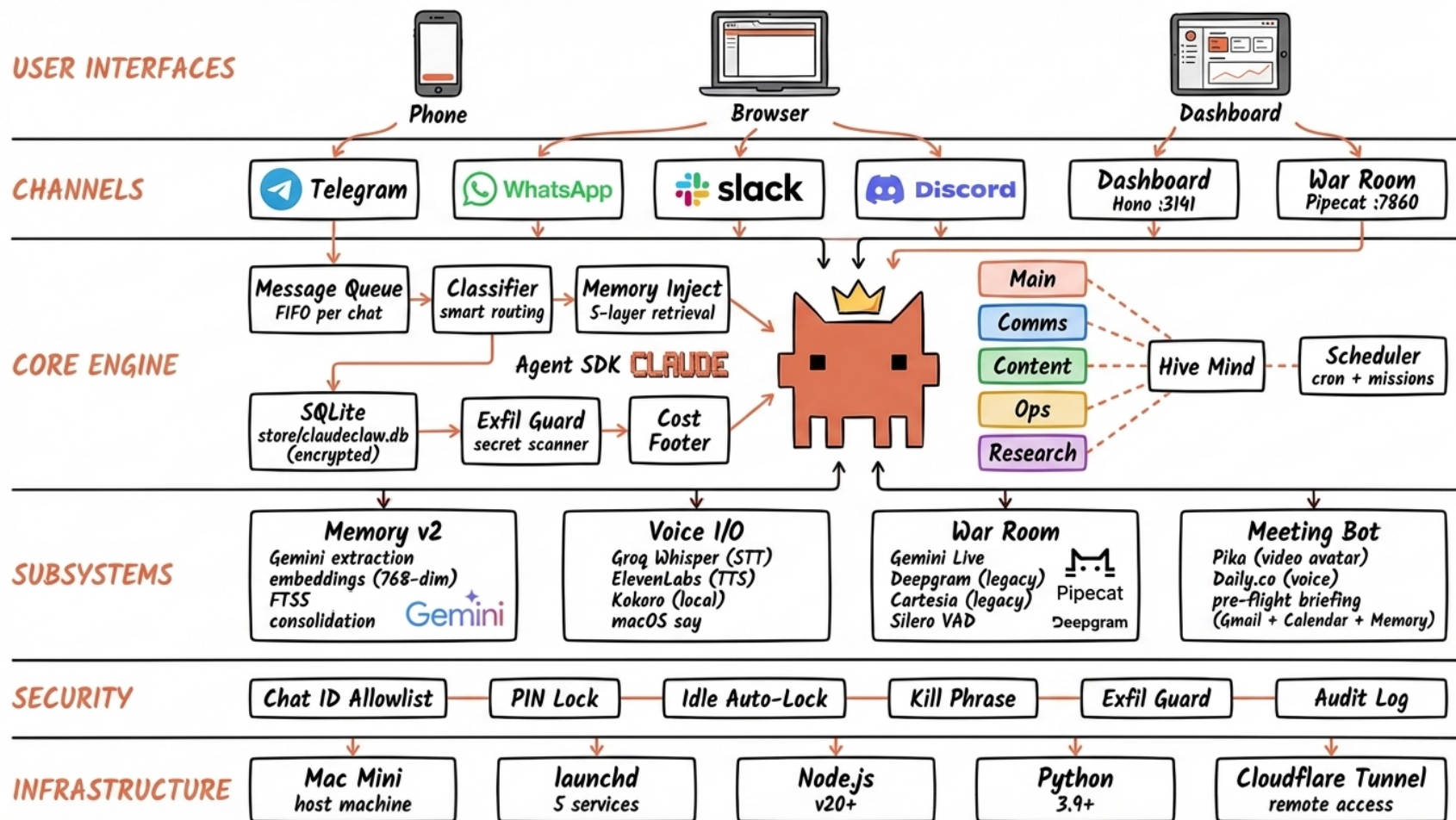
v0: One Bridge, One Bot, One Brain



v0 — one bridge, one bot, one brain

The original system. Four sparse layers: one phone, one Telegram channel, a simple 7-stage pipeline with SQLite memory, and a Mac Mini running Node.js with 200 lines of code. Most of the space is empty.

v2: The Full Operating System



Six dense layers. Multiple user interfaces. Six channels (Telegram, WhatsApp, Slack, Discord, Dashboard, War Room). A core engine with five agents, hive mind, scheduler, and an 8-stage pipeline. Four subsystems (Memory v2, Voice, War Room, Meeting Bot). A security bar. Mac Mini, launchd, Node.js, Python, and Cloudflare Tunnel.

Want the full repo with all features and updates?

Join the Early AI Dopters community

<https://www.skool.com/earlyaidopters/about>